

Process Creation, Monitoring, and CPU Affinity in Linux

Objective

The objective of this experiment is to understand process creation, process monitoring, CPU utilization, and CPU core affinity in Linux using basic system commands.

Program Description

A C program named `while1.c` containing an infinite loop was compiled and executed multiple times in the background. This program was used to observe how Linux handles multiple processes and CPU scheduling.

Steps Performed and Observations

Step 1: Compilation and Execution

The program was compiled using the `gcc` compiler. After compilation, the executable was run in the background using the ampersand symbol, which allows the shell to accept further commands while the process runs.

Command used:

```
gcc 01_while1.c -o while1
./while1 &
```

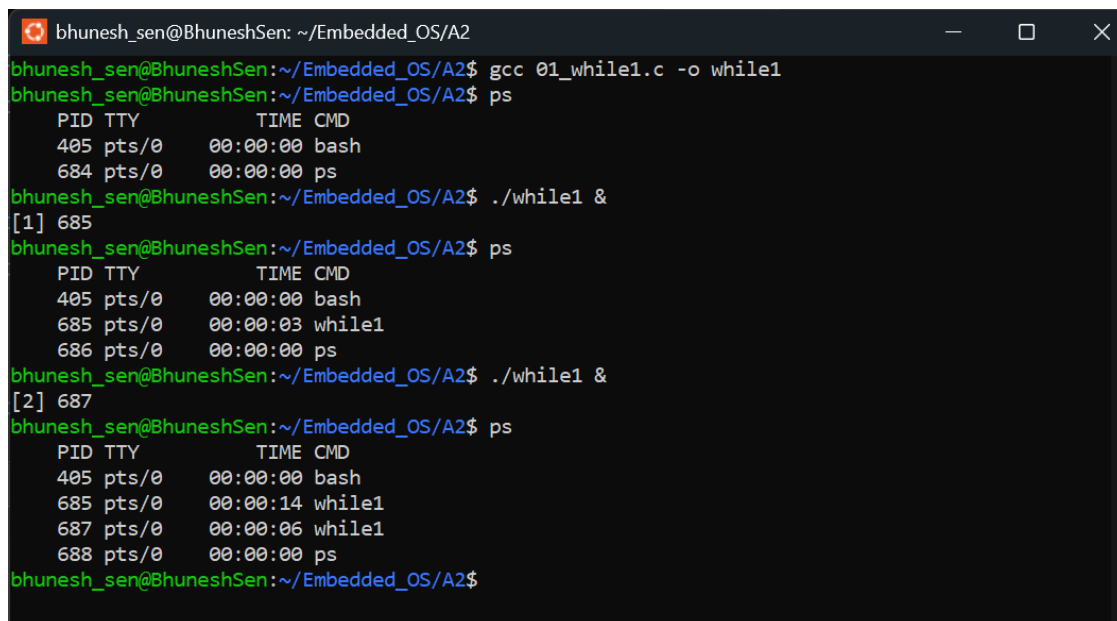
Each execution of the program created a new process with a unique process ID (PID).

Step 2: Process Monitoring using `ps`

The `ps` command was used to list currently running processes in the terminal session.

Command used:

```
ps
```



```

bhunesh_sen@BhuneshSen: ~/Embedded_OS/A2
bhunesh_sen@BhuneshSen:~/Embedded_OS/A2$ gcc 01_while1.c -o while1
bhunesh_sen@BhuneshSen:~/Embedded_OS/A2$ ps
  PID TTY          TIME CMD
   405 pts/0        00:00:00 bash
   684 pts/0        00:00:00 ps
bhunesh_sen@BhuneshSen:~/Embedded_OS/A2$ ./while1 &
[1] 685
bhunesh_sen@BhuneshSen:~/Embedded_OS/A2$ ps
  PID TTY          TIME CMD
   405 pts/0        00:00:00 bash
   685 pts/0        00:00:03 while1
   686 pts/0        00:00:00 ps
bhunesh_sen@BhuneshSen:~/Embedded_OS/A2$ ./while1 &
[2] 687
bhunesh_sen@BhuneshSen:~/Embedded_OS/A2$ ps
  PID TTY          TIME CMD
   405 pts/0        00:00:00 bash
   685 pts/0        00:00:14 while1
   687 pts/0        00:00:06 while1
   688 pts/0        00:00:00 ps
bhunesh_sen@BhuneshSen:~/Embedded_OS/A2$
```

Observation:

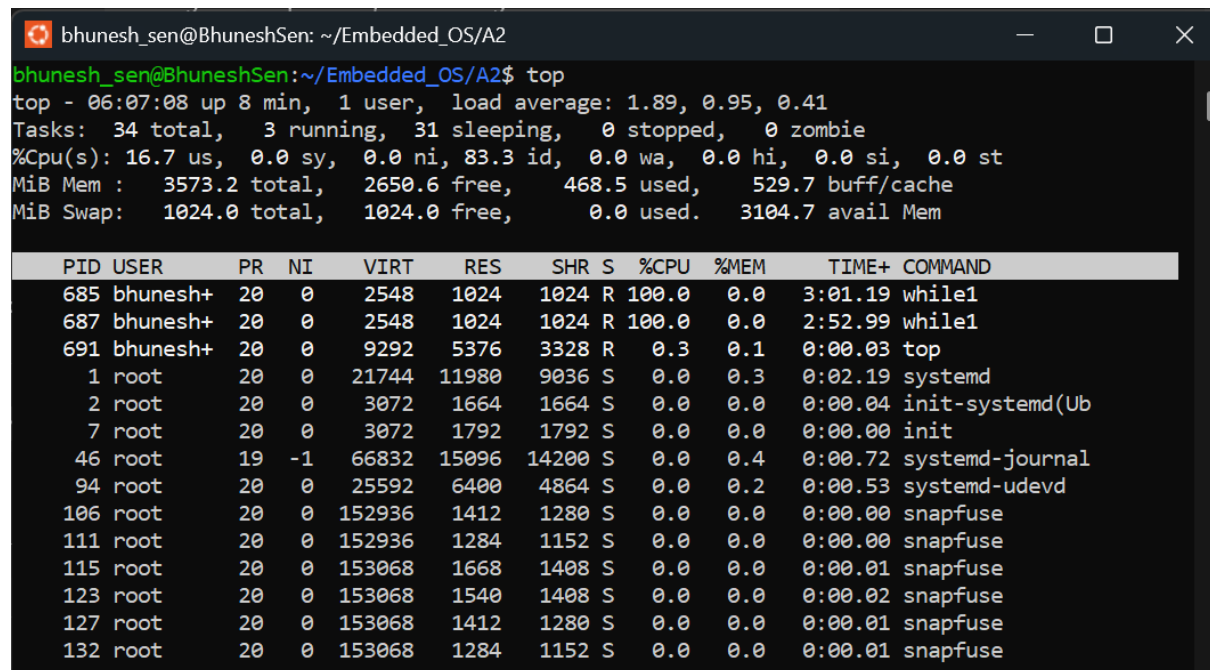
Multiple instances of the while1 program were visible, each with a different PID. This confirms that every execution of the same executable creates a separate process.

Step 3: CPU Utilization using top

The top command was used to monitor CPU usage in real time.

Command used:

top



```
bhunesh_sen@BhuneshSen: ~/Embedded_OS/A2
bhunesh_sen@BhuneshSen:~/Embedded_OS/A2$ top
top - 06:07:08 up 8 min, 1 user, load average: 1.89, 0.95, 0.41
Tasks: 34 total, 3 running, 31 sleeping, 0 stopped, 0 zombie
%Cpu(s): 16.7 us, 0.0 sy, 0.0 ni, 83.3 id, 0.0 wa, 0.0 hi, 0.0 si, 0.0 st
MiB Mem : 3573.2 total, 2650.6 free, 468.5 used, 529.7 buff/cache
MiB Swap: 1024.0 total, 1024.0 free, 0.0 used, 3104.7 avail Mem

  PID USER      PR  NI   VIRT   RES   SHR  S  %CPU  %MEM    TIME+  COMMAND
 685 bhunesh+  20   0   2548   1024   1024  R 100.0   0.0   3:01.19 while1
 687 bhunesh+  20   0   2548   1024   1024  R 100.0   0.0   2:52.99 while1
 691 bhunesh+  20   0   9292   5376   3328  R   0.3   0.1   0:00.03 top
    1 root      20   0  21744  11980   9036  S   0.0   0.3   0:02.19 systemd
    2 root      20   0   3072   1664   1664  S   0.0   0.0   0:00.04 init-systemd(Up
    7 root      20   0   3072   1792   1792  S   0.0   0.0   0:00.00 init
   46 root      19  -1  66832  15096  14200  S   0.0   0.4   0:00.72 systemd-journal
   94 root      20   0  25592   6400   4864  S   0.0   0.2   0:00.53 systemd-udevd
  106 root      20   0  152936  1412   1280  S   0.0   0.0   0:00.00 snapfuse
  111 root      20   0  152936  1284   1152  S   0.0   0.0   0:00.00 snapfuse
  115 root      20   0  153068  1668   1408  S   0.0   0.0   0:00.01 snapfuse
  123 root      20   0  153068  1540   1408  S   0.0   0.0   0:00.02 snapfuse
  127 root      20   0  153068  1412   1280  S   0.0   0.0   0:00.01 snapfuse
  132 root      20   0  153068  1284   1152  S   0.0   0.0   0:00.01 snapfuse
```

Observation:

Each while1 process was consuming nearly 100 percent CPU. The process state was shown as Running. This high CPU usage occurred because the program contains an infinite loop without any delay or sleep function.

Step 4: Checking Number of CPU Cores

The total number of available CPU cores in the system was checked.

Command used:

nproc

Observation:

The system contains 12 CPU cores.

Step 5: Setting CPU Affinity using taskset

Specific CPU cores were assigned to the while1 processes using the taskset command.

Command used:

taskset -c 0 ./while1 &

taskset -c 1 ./while1 &

Observation:

One instance of the program was restricted to run only on CPU core 0, while another instance was restricted to CPU core 1. This demonstrates CPU affinity, where a process is bound to specific processor cores.

Step 6: Verifying CPU Core Assignment

The ps command was used with additional options to verify the CPU core on which each process was running.

Command used:

```
ps -o pid,psr,comm -C while1
```

Observation:

The PSR column displayed the processor number on which each process was executing. The output confirmed that the processes were running on the assigned CPU cores.

```

bhunesh_sen@BhuneshSen: ~/Embedded_OS/A2
bhunesh_sen@BhuneshSen:~/Embedded_OS/A2$ nproc
12
bhunesh_sen@BhuneshSen:~/Embedded_OS/A2$ taskset -c 0 ./while1 &
[2] 744
bhunesh_sen@BhuneshSen:~/Embedded_OS/A2$ taskset -c 1 ./while1 &
[3] 745
bhunesh_sen@BhuneshSen:~/Embedded_OS/A2$ ps -o pid,psr,comm -C while1
  PID PSR COMMAND
   741   0 while1
   744   0 while1
   745   1 while1
bhunesh_sen@BhuneshSen:~/Embedded_OS/A2$ ps
  PID TTY          TIME CMD
   405 pts/0    00:00:00 bash
top - 06:17:14 up 18 min,  1 user,  load average: 3.06, 2.02, 1.08
Tasks:  3 total,   3 running,   0 sleeping,   0 stopped,   0 zombie
%Cpu(s): 16.7 us,  0.1 sy,   0.0 ni, 83.2 id,   0.0 wa,   0.0 hi,   0.0 si,   0.0 st
MiB Mem : 3573.2 total, 2645.5 free,  466.0 used,  537.7 buff/cache
MiB Swap: 1024.0 total, 1024.0 free,   0.0 used. 3107.2 avail Mem

  PID USER      PR  NI   VIRT   RES   SHR S  %CPU  %MEM    TIME+  COMMAND
   745 bhunesh+  20   0   2548   1024   1024 R   100.0   0.0   4:17.76 while1
   741 bhunesh+  20   0   2548   1024   1024 R    51.3   0.0   2:32.15 while1
   744 bhunesh+  20   0   2548   1024   1024 R    51.0   0.0   2:17.21 while1

```